

Knowledge Based Engineering (KBE) AE4204

Dr.ir. G. La Rocca

Flight Performance & Propulsion Group (FPP)

Lecture 6

- *Rules in KBE*
- *Product model and Generative modeling*



Content of this lecture

- **Rules in KBE**

A proposed categorization of the engineering rules that capture domain knowledge and drive the generation of a design:

- Logic rules
- Mathematical rules
- Geometry rules
- Configuration rules
- Integration rules

- Definition of ***product model*** and ***generative modeling***

- Recap on KBE vs. CAD

KBE rules:

- *Logic rules*
- *Math rules*
- *Geometry rules*
- *Configuration rules*
- *Integration rules*

Capturing domain knowledge with KBE

Logic rules (or conditional rules)

- In engineering models, logic rules are typically used to enforce **design decisions**, **regulatory constraints**, or **technology choices**.

```
if input_color == "red":  
    print("my favorite!")  
else:  
    print("c-mon dude!")
```

```
if input_color == "red":  
    print("my favorite!")  
else:  
    if input_color == "yellow" or in  
        print("flashy!")  
    else:  
        print("c-mon dude!")
```

default

```
if input_color == "red":  
    print("my favorite!")  
  
elif input_color == "yellow":  
    print("flashy!")  
  
elif input_color == "green":  
    print("boring!")
```

```
else:  
    print("c-mon dude!")
```

default

Capturing domain knowledge with KBE

Logic rules (or conditional rules) continued

- Alternative type of conditional rules can be defined using **dictionaries** and `.get()`
- Dictionaries provide a compact way to implement lookup tables, which are very common in engineering knowledge capture, for example when mapping design categories to predefined design choices

Dictionary syntax:

```
dictionary_name = {key1: value1, key2: value2, key_n: value_n}
```

```
def color_consideration(colorkey):  
    my_options = {"red": "my favourite",  
                  "green": "boring",  
                  "yellow": "flashy"}  
    return my_options.get(colorkey, "c-mon dude!")
```

default

Makes a consideration according to the value of the colour attribute provided as input

Capturing domain knowledge with KBE

Math rules

- Include any kind of mathematical rule (e.g. trigonometric functions and operators for matrices and vectors algebra)
- Mathematical operator such as +, -, *, ** are implemented as Python functions, hence directly usable in KBE rules
- Other specific functions and operators can be defined using the given KBE language.

Example: The mathematical expression $L = \frac{1}{2} \rho V^2 \cdot S \cdot C_L$ maps into:

```
Wing_lift = 0.5 * rho * (velocity **2) * surface * wing_lift_coefficient
```

Mathematical rules can be used both in the antecedent and consequent part of any IF-THEN-like rule.

Capturing domain knowledge with KBE

Geometry rules

- They define how geometric entities are generated or manipulated **as a function of model parameters**.
- In a KBE model, geometry is not created manually but generated **programmatically** through rules that **connect to CAD libraries**

@Part

```
def container(self):  
    return Box(length=10,  
               width=20,  
               height=30)
```

KBE language
geometry
primitives

Generate a box of
dimensions 10X20X30

@Part

```
def wing(self):  
    return LoftedSurface(profiles = [self.airfoil1,  
                                     self.mid_airfoil,  
                                     self.airfoil2])
```

Generate a smooth
surface interpolating
on previously
generated curves

Capturing domain knowledge with KBE

Geometry rules (continued)

- These rules combine geometric reasoning with mathematical relations and are very common in **parametric design**

```
@Attribute
```

```
def position_pointA(self):  
    return self.position_pointB + self.translation_vector
```

```
@Attribute
```

```
def diameter_hole(self):  
    return self.diameter_pin + self.clearance_value
```

Geometry-handling rules are generally absent in classical knowledge-based systems, because they require spatial reasoning and geometric representations of assemblies, parts, and features.

Capturing domain knowledge with KBE

Configuration (or topology) rules

- Configuration rules determine the **structure or topology of the product**, for example **how many** components are instantiated ...

```
@Part  
def my_beam(self):  
    return Beam(quantify=3 if (self.total_load > 100) else 2)
```



The number of *Beam* instantiations is computed based on the evaluation of a logic rule.

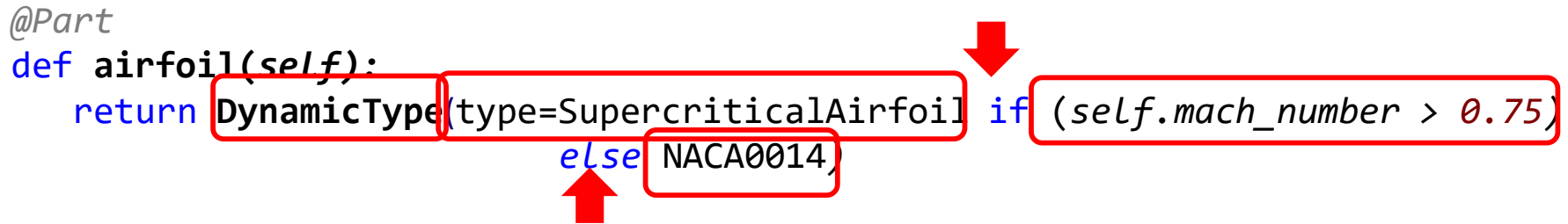
A mathematical expression is computed to evaluate the antecedent of the IF-THEN logic rule.

Capturing domain knowledge with KBE

Configuration (or topology) rules

- Configuration rules determine the **structure or topology of the product**, for example how many components are instantiated or **which component type** is selected.

```
@Part
def airfoil(self):
    return DynamicType(type=SupercriticalAirfoil if (self.mach_number > 0.75)
                       else NACA0014)
```



The class to be instantiated to create the object *airfoil* is selected based on a logic rule combined with a mathematical expression

DynamicType allows the system to automatically switch between different design solutions or even product architectures, depending on conditions.

Capturing domain knowledge with KBE

Configuration (or topology) rules

```
@Part
def my_beam(self):
    return Beam(quantify=round(self.total load/self.allowable load),
                beam_length=10 if (child.index==1) else 8)
```

The number of *Beam* instantiations is computed based on the evaluation of mathematical expression.

The attribute *beam_length* of each individual instance is evaluated based on a logic rule

→ when multiple instances are generated using **quantify**, they are **not “carbon copies”**. Each instance can be individually parameterized and controlled

Capturing domain knowledge with KBE

Configuration (or topology) rules

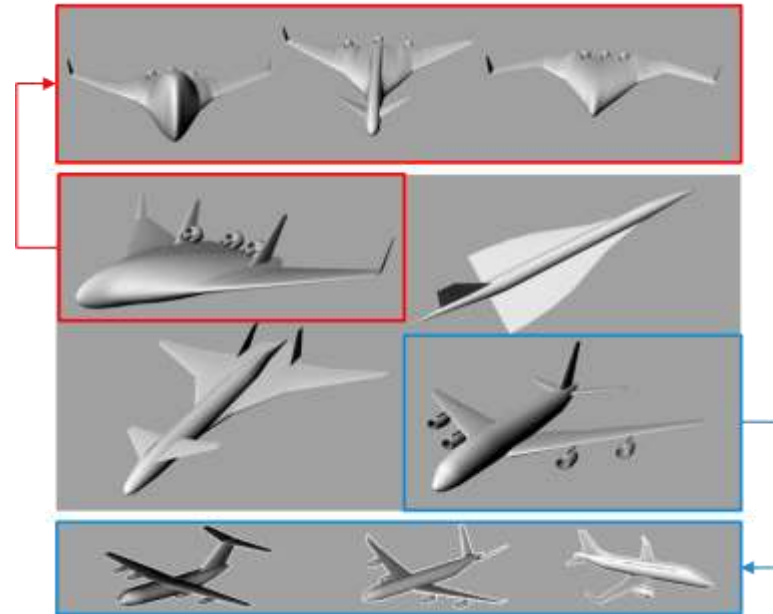
To recap:

Configuration rules define the topology of the product by determining:

- **which** components are present
- **how many** instances are created
- **how** they are **individually** arranged in the assembly

Geometry rules change **shapes**, configuration rules change the **product architecture**

Aircraft instances produced by a single KBE model



Good to keep in mind...

1. Classical knowledge-based systems typically handle symbolic rules but **do not manipulate geometry**.
2. Parametric CAD systems can generate and modify geometry, but their **rule capabilities are limited**.
3. KBE systems **combine logic rules, mathematical rules, configuration rules, and geometry rules** in a single framework.

Capturing domain knowledge with KBE

Integration rules

- Integration rules control the interaction between the KBE model and **external tools**, **databases**, or **file formats**. This enables the automation of complete engineering workflows

```
@Attribute
def writer(self):
    return STEPWriter(trees=[self],
                      filename="path/to/your/file.stp")
```

```
@Part
def imported_geometry(self):
    return STLReader(filename=self.filename)
```

```
@Attribute
def xfoil_analysis(self):
    return run_xfoil(self.section_points_xfoil,
                     self.reynolds_number,
                     self.alpha)
```

Capturing domain knowledge with KBE

Mesh Generation rules

```
class PlateMesh(GeomBase):
```

```
    """ """
    pitch = Input(2)
```

```
    def no_elements(self, edges):
        ...
```

```
    @Part
```

```
    def plate(self):
        return RectangularFace(width=10, length=20)
```

```
    @Attribute
```

```
    def edge_chain(self):
        ...
```

```
    @Part
```

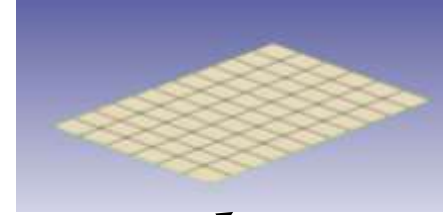
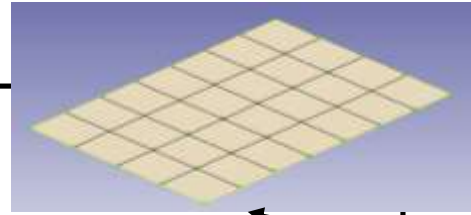
```
    def edge_control(self, child):
        return FixedNumber(quantify=len(self.edge_chain),
                           shape=self.edge_chain[child.index],
                           no=self.no_elements(child.shape))
```

```
    @Part
```

```
    def face_control(self):
        return Quad(shape=self.plate)
```

```
    @Part
```

```
    def grid(self):
        return Mesh(shape_to_mesh=self.plate,
                    controls=self.edge_control + [self.face_control])
```



Effect of varying the maximum pitch size of each element on meshing

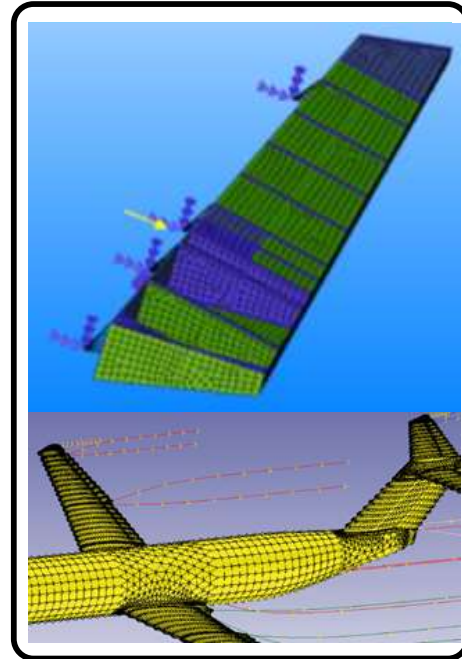
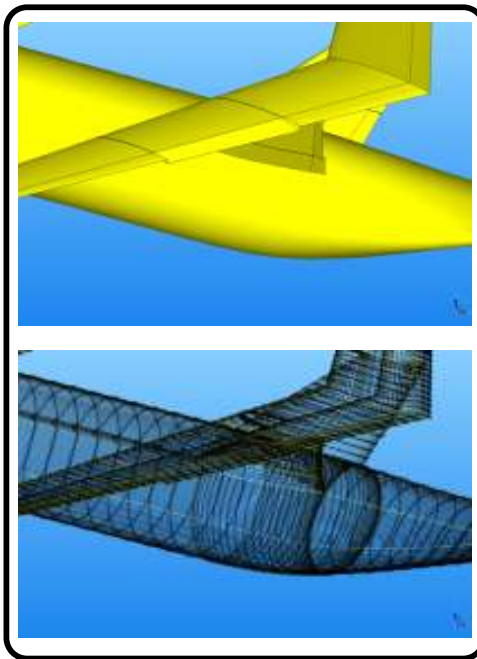
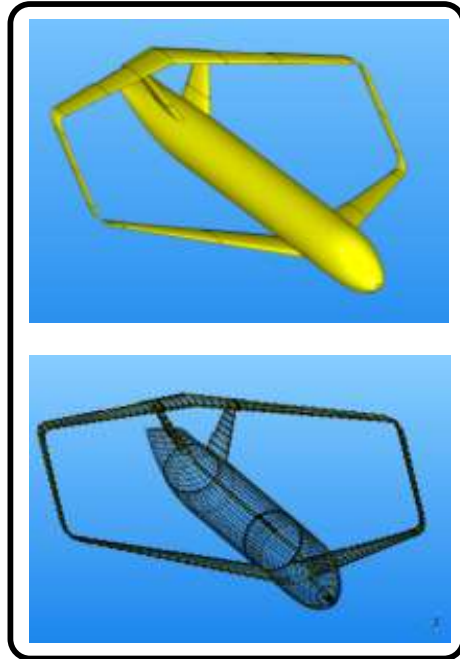
Determines the number of nodes on each edge, based on standard pitch criterion

Ensures quadrilateral elements throughout the plate

Generates the mesh based on the provided mesh controls

Capturing domain knowledge with KBE

Mesh Generation rules (structured, unstructured, hybrid, surface, volume)



Capturing domain knowledge with KBE

Rules recap

<i>Rule type</i>	<i>Purpose</i>
Logic rules	→ decision making
Mathematical rules	→ compute values
Geometry rules	→ generate geometry
Configuration rules	→ define product topology
Integration rules	→ connect to external tools and data

Generative models for generative design

KBE product model and generative design

- A KBE application contains a codified representation of the engineering knowledge required to design and configure a class of products.
- This codified representation is called a ***product model***
- The product model therefore consists of the **complete set of rules** (logic, math, geometry, configurations) that determine how the product is designed, possibly supported by external tools integrated by means of **integration rules**



KBE product model and generative design

A product model operates as follows:

1. input parameters describing a design instance are provided
2. the KBE system applies the engineering rules encoded in the model
3. The design is generated, with little or no human intervention.



This capability is known as ***generative design***, thereby the product model is also called ***generative model***.

KBE product model and generative design

The goal of KBE is **to capture how a product should be designed**, not only the final geometry

"if you have to bake a cake, the picture of the cake is not sufficient. You need the recipe!"

Similarly:

- a **CAD model** represents **one instance** of a design
- a **product model** contains **the rules that allow the design to be generated** automatically.

Carrot Cake Ingredients:

Cake:

4 eggs		2 teaspoons baking powder
2 cups sugar		1 cup pecans, chopped
4 cups grated carrots	1 cup vegetable oil	
2 cup flour		2 teaspoons baking soda
2 teaspoons cinnamon		

Cream Cheese Frosting:

1/2 stick butter, softened	8 oz. cream cheese, softened
1 box powdered sugar (1 lb.)	2 teaspoons vanilla extract

Directions:

Carrot Cake Directions:

1. Combine sugar, baking powder, flour, cinnamon and baking soda in a bowl. Add in the eggs and oil.
2. Mix until just blended and then add the carrots and pecans.
3. Bake at 375 degrees F to 400 degrees F for 35 to 40 minutes or until done.

Frosting Directions:

1. Mix cream cheese and butter until smooth, add sugar and vanilla and blend well. Spread on cooled cake.

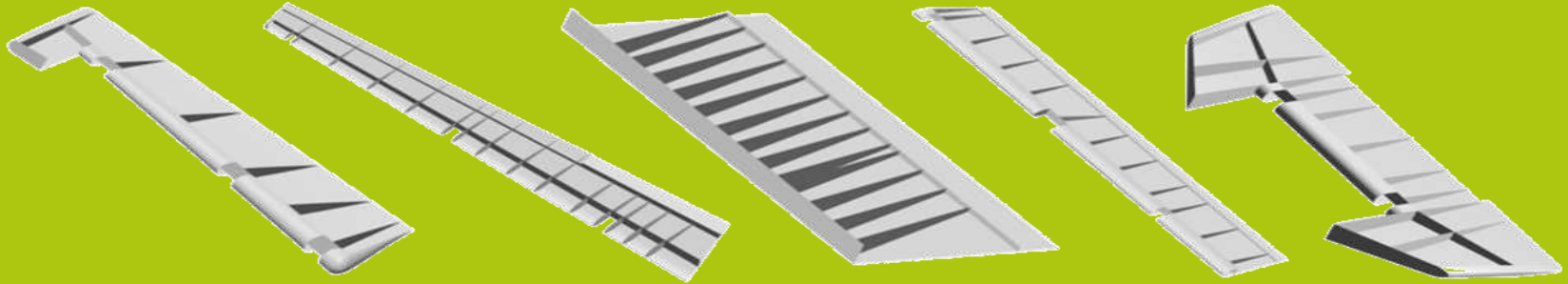


KBE product model and generative design

EXAMPLE:

- A detailed CAD model of an aircraft movable does not represent the design process and the knowledge required to generate such a movable
- In KBE, it is the final result obtained from a specific *instantiation* of the knowledge owned by the movable design specialist.

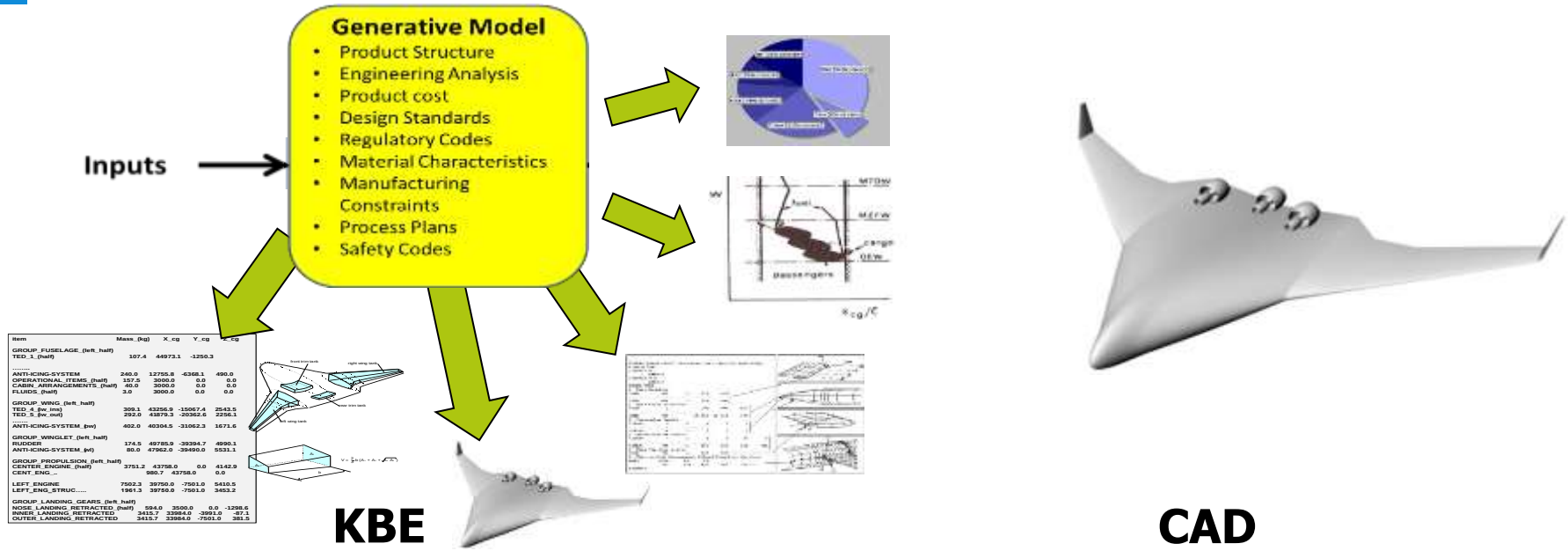
Geometry output produced by an aircraft movables generative model



*Movables upper skin removed to show internal structure

KBE product model and generative design

While CAD focuses on handling and delivering geometry, KBE focus on *rules and knowledge capturing*, with *geometry* being just one of the many output that can be generated.



Main reference material for this lecture

1. **Book** *MDO supported by KBE* by Wiley: **Chapter 9 on KBE**
2. **Journal paper** *Knowledge based engineering: Between AI and CAD. Review of a language based technology to support engineering design,*
G. La Rocca,, Journal of Advanced Engineering Informatics, Vol. 26, pp.159–179, 2012.
3. **ParaPy documentation**